

基于 YOLOv11 的目标检测智能体 — 环境安装配置教程

适用项目：基于 YOLOv11 与智能体架构的遥感目标检测平台（rsod-agent-platform）

技术栈概览：Python 3.10+ / FastAPI / LangChain / LangGraph / YOLOv11 / Vue 3 / Vite / Element Plus / PostgreSQL / Redis / MinIO / Docker Compose

目录

- [一、基础环境安装](#)
 - [1.1 Git 与 GitHub 版本控制](#)
 - [1.2 Python 安装与虚拟环境](#)
 - [1.3 Node.js 安装](#)
- [二、IDE 安装与配置](#)
 - [2.1 VS Code 安装与配置（推荐）](#)
 - [2.2 PyCharm 安装与配置（备选）](#)
- [三、Docker 与 Docker Compose 部署](#)
 - [3.1 Docker Desktop 安装](#)
 - [3.2 Docker Compose 项目服务编排](#)
 - [3.3 Docker Compose 常用命令](#)
- [四、项目初始化与运行](#)
- [五、环境验证清单](#)
- [六、常见问题排查](#)

一、基础环境安装

1.1 Git 与 GitHub 版本控制

Git 是项目协作开发的版本控制工具，所有代码通过 Git + GitHub 进行管理和协作。

1.1.1 安装 Git

Windows:

1. 访问 Git 官网下载页面: <https://git-scm.com/download/win>
2. 下载 **64-bit Git for Windows Setup**
3. 运行安装程序，保持默认选项即可，以下几点注意：

- **Adjusting your PATH environment** → 选择 `Git from the command line and also from 3rd-party software` (推荐)
- **Choosing the default editor** → 可选择 VS Code 作为默认编辑器
- **Configuring the line ending conversions** → 选择 `Checkout Windows-style, commit Unix-style line endings`

4. 安装完成后, 右键菜单会出现 "Git Bash Here" 选项

macOS:

```
# 方法 1: 使用 Homebrew (推荐)
brew install git

# 方法 2: 使用 Xcode Command Line Tools
xcode-select --install
```

Linux (Ubuntu/Debian) :

```
sudo apt update
sudo apt install git -y
```

验证安装:

```
git --version
# 期望输出: git version 2.x.x
```

1.1.2 Git 基础配置

安装完成后, 需要配置你的身份信息, 这会出现每次 Git 提交记录中:

```
# 设置用户名 (建议使用真实姓名拼音或英文名)
git config --global user.name "你的名字"

# 设置邮箱 (建议与 GitHub 注册邮箱一致)
git config --global user.email "your.email@example.com"

# 设置默认分支名称为 main
git config --global init.defaultBranch main

# 查看当前配置
git config --list
```

1.1.3 GitHub 账号与 SSH 密钥配置

第一步: 注册 GitHub 账号

访问 <https://github.com/signup>, 使用邮箱注册并完成验证。

第二步: 生成 SSH 密钥

```
# 检查是否已有 SSH 密钥
ls -al ~/.ssh

# 如果没有 id_ed25519.pub 文件，则生成新的 SSH 密钥
ssh-keygen -t ed25519 -C "your.email@example.com"
# 一路回车即可（也可以设置密码短语增加安全性）
```

第三步：将公钥添加到 GitHub

```
# Windows (Git Bash 中执行)
cat ~/.ssh/id_ed25519.pub

# macOS
pbcopy < ~/.ssh/id_ed25519.pub    # 直接复制到剪贴板

# Linux
xclip -sel copy < ~/.ssh/id_ed25519.pub    # 需要安装 xclip
```

然后：

1. 打开 <https://github.com/settings/keys>
2. 点击 **New SSH key**
3. Title 填写一个描述性名称（如 "我的笔记本"）
4. Key type 选择 **Authentication Key**
5. 将刚才复制的公钥内容粘贴到 Key 框中
6. 点击 **Add SSH key**

第四步：验证连接

```
ssh -T git@github.com
# 如果看到：Hi xxx! You've successfully authenticated, but GitHub does not provide shell
access.
# 说明配置成功
```

首次连接提示：会询问 `Are you sure you want to continue connecting (yes/no)?`，输入 `yes` 回车即可。

1.1.4 Git 常用命令速查

```
# 克隆远程仓库到本地
# HTTPS方式（推荐）
https://github.com/your-username/rsod-web-platform.git

# SSH方式（需要配置SSH密钥，参见上一步）
git@github.com:your-username/rsod-web-platform.git

# 查看当前状态（最常用的命令）
```

```
git status

# 创建并切换到新分支
git checkout -b feature/your-feature-name

# 添加文件到暂存区
git add .                # 添加所有修改
git add backend/main.py  # 添加指定文件

# 提交更改（写清楚本次修改的内容）
git commit -m "feat: 添加目标检测API接口"

# 推送本地分支到远程
git push origin feature/your-feature-name

# 拉取远程仓库最新代码
git pull origin main

# 查看所有分支
git branch -a

# 查看提交历史
git log --oneline
```

分支命名建议：

前缀	用途	示例
feature/	新功能	feature/detection-api
fix/	Bug 修复	fix/login-error
docs/	文档更新	docs/api-guide
refactor/	代码重构	refactor/agent-router

1.2 Python 安装与虚拟环境

Python 是本项目后端开发、模型推理的核心语言。本项目要求 **Python 3.10** 及以上版本。

1.2.1 安装 Python

Windows：

- 访问 Python 官网下载页面：<https://www.python.org/downloads/>
- 下载 Python 3.10.x 或 3.11.x 版本（推荐 3.10.20 或 3.11.9）
-  **重要：安装时务必勾选 "Add Python to PATH"**
- 点击 "Install Now" 完成安装

macOS：

```
# 方法 1: 使用 Homebrew (推荐)
brew install python@3.10

# 方法 2: 使用 pyenv 管理多版本 (适合需要多个 Python 版本的场景)
brew install pyenv
pyenv install 3.10.20
pyenv global 3.10.20
```

Linux (Ubuntu/Debian) :

```
sudo apt update
sudo apt install python3.10 python3.10-venv python3-pip -y
```

验证安装:

```
python --version      # Windows
python3 --version     # macOS / Linux
# 期望输出: Python 3.10.x 或 Python 3.11.x

pip --version         # Windows
pip3 --version        # macOS / Linux
# 期望输出: pip 23.x.x 或更高
```

1.2.2 创建与管理虚拟环境

虚拟环境用于隔离项目依赖，避免不同项目之间的包版本冲突。每个项目都应该使用独立的虚拟环境。

创建虚拟环境:

```
# 进入项目根目录
cd rsod-agent-platform/backend

# 创建虚拟环境 (推荐命名为 .venv)
python -m venv .venv      # Windows
python3.10 -m venv .venv  # macOS / Linux
```

激活虚拟环境:

```
# Windows (CMD)
.venv\Scripts\activate

# Windows (PowerShell)
.venv\Scripts\Activate.ps1

# macOS / Linux
source .venv/bin/activate

#查看当前环境 python 路径
which python
```

激活成功后，终端提示符前会出现 `(.venv)` 标识：

```
(.venv) C:\Users\xxx\rsod-agent-platform>      # Windows  
(.venv) user@hostname:~/rsod-agent-platform$    # macOS / Linux
```

退出虚拟环境：

```
deactivate
```

 注意：后续所有 `pip install` 操作都必须在虚拟环境激活状态下执行。

1.2.3 配置 pip 镜像源（国内用户）

由于网络原因，国内直接访问 PyPI 速度较慢，建议配置清华镜像源：

```
# 临时使用（单次安装）  
pip install <package> -i https://mirrors.aliyun.com/pypi/simple  
  
# 永久配置（推荐）  
pip config set global.index-url https://mirrors.aliyun.com/pypi/simple  
pip config set global.trusted-host mirrors.aliyun.com
```

1.2.4 项目核心 Python 依赖一览

以下是本项目后端使用的主要 Python 包（完整清单见项目中的 `backend/requirements.txt`）：

类别	包名	版本要求	用途
Web 框架	fastapi	0.110.0	异步 API 服务框架
	uvicorn[standard]	0.24.0.post1	ASGI 服务器，运行 FastAPI
	python-multipart	0.0.9	支持文件上传
智能体框架	langchain	0.3.0	Agent 基础能力、工具定义、Prompt 管理
	langchain-openai	0.2.0	OpenAI 模型接入
	langchain-community	0.3.0	LangChain 社区集成
	langgraph	0.3.0	多 Agent 工作流编排、状态管理
	openai	1.50.0	OpenAI API SDK
	ollama	0.3.0	本地 LLM 备选方案
	ultralytics	8.3.0	YOLOv11 模型推理与训练
目标检测	opencv-python	4.8.0.76	图像/视频读取与处理
	Pillow	10.0.0	图像格式转换
	sqlalchemy	2.0.0	ORM 框架
数据库	psycopg2-binary	2.9.9	PostgreSQL 连接驱动
	pgvector	0.3.0	PostgreSQL 向量存储扩展（RAG 用）
	alembic	1.13.0	数据库迁移工具
缓存	redis	5.0+	Redis 客户端，缓存 + 对话历史
对象存储	minio	7.2+	MinIO 客户端，图片/模型文件存储
认证安全	python-jose[cryptography]	3.3.0	JWT 用户认证
	passlib[bcrypt]	1.7.4	密码哈希加密
数据验证	pydantic	2.8.0	请求/响应数据验证
	pydantic-settings	2.4.0	配置管理
工具库	httpx	0.27.0	异步 HTTP 客户端
	python-dotenv	1.0.0	环境变量加载

```
# backend/requirements.txt

# =====
```

```
# 基于 YOLOv11 的目标检测智能体平台 — Python 依赖
# =====

# — Web 框架 —
fastapi==0.110.0 # Web应用核心框架, 提供异步API路由、请求验证等
uvicorn[standard]==0.24.0.post1 # ASGI服务器, 用于运行FastAPI应用
python-multipart==0.0.9 # 文件上传支持, FastAPI处理multipart/form-data必需

# — 智能体框架 (LangChain + LangGraph) —
langchain==0.3.0 # Agent 基础能力、Prompt 管理
langchain-openai==0.2.0 # OpenAI 模型接入
langchain-community==0.3.0 # 社区集成 (Redis 对话历史等)
langgraph==0.2.0 # 多Agent工作流编排、图形化管理
openai==1.50.0 # OpenAI API SDK

# — 本地 LLM 备选 —
ollama==0.3.0 #本地 LLM 模型备选

# — 目标检测 (YOLOv11) —
ultralytics==8.3.0 # Ultralytics YOLO框架 (包含PyTorch)
opencv-python==4.8.0.76 # OpenCV图像处理库
Pillow==10.0.0 # Pillow图像处理库

# — 数据库 (PostgreSQL + ORM) —
sqlalchemy==2.0.0 # SQLAlchemy ORM框架
psycopg2-binary==2.9.9 # PostgreSQL数据库驱动
pgvector==0.3.0 #PostgreSQL 向量存储扩展 (RAG 用)

alembic==1.13.0 # Alembic 数据库迁移框架

# — 缓存 (Redis) —
redis==5.0.0

# — 对象存储 (MinIO) —
minio==7.2.0 # 图片/模型文件存储

# — 认证安全 —
python-jose[cryptography]==3.3.0 #JWT 用户认证
passlib[bcrypt]==1.7.4 #密码哈希加密

# — 数据验证与配置 —
pydantic==2.8.0 # 请求/响应数据验证
pydantic-settings==2.4.0 # 配置管理

# — 工具库 —
httpx==0.27.0 # 异步 HTTP 客户端
python-dotenv==1.0.0 # 环境变量管理
```

安装依赖:

```
# 确保虚拟环境已激活, 然后安装所有依赖
```



```
pip install -r requirements.txt

# 如遇版本冲突安装失败，修改对应版本号，执行下列操作
# 1. 停用虚拟环境
deactivate
# 2. 删除旧的虚拟环境
rm -rf .venv

# 3. 使用 Python 3.10+ 创建新虚拟环境
python3.10 -m venv .venv

# 4. 激活虚拟环境
source .venv/bin/activate

# 5. 升级 pip
pip install --upgrade pip

# 6. 重新安装所有依赖
pip install -r requirements.txt
```

验证核心包安装成功：

```
python -c "import fastapi; print(f'FastAPI {fastapi.__version__} ✓')"
```

```
python -c "import uvicorn; print(f'Uvicorn {uvicorn.__version__} ✓')"
```

```
python -c "import langchain; print(f'LangChain {langchain.__version__} ✓')"
```

```
python -c "import langgraph; print(f'LangGraph {langgraph.__version__} ✓')"
```

```
python -c "import ultralytics; print(f'Ultralytics {ultralytics.__version__} ✓')"
```

```
python -c "import cv2; print(f'OpenCV {cv2.__version__} ✓')"
```

```
python -c "import sqlalchemy; print(f'SQLAlchemy {sqlalchemy.__version__} ✓')"
```

1.3 Node.js 安装

Node.js 是前端 Vue 3 项目的运行环境，用于依赖管理（npm）和开发服务器（Vite）。本项目要求 **Node.js 18** 及以上版本。

1.3.1 安装 Node.js

Windows / macOS：

1. 访问 Node.js 官网：<https://nodejs.org/>
2. 下载 **LTS**（长期支持）版本（24.x）
3. 运行安装程序，保持默认选项即可

macOS（使用 Homebrew）：

```
brew install node@24
```

验证安装：

```
node --version
# 期望输出: v24.x.x

npm --version
# 期望输出: 11.x.x
```

1.3.2 配置 npm 镜像源（国内用户）

```
# 设置淘宝镜像
npm config set registry http://registry.npm.taobao.org

# 验证配置
npm config get registry
# 期望输出: http://registry.npm.taobao.org
```

1.3.3 项目核心前端依赖一览

以下是本项目前端使用的主要 npm 包（完整清单见项目中的 `frontend/package.json`）：

类别	包名	版本要求	用途
框架	vue	3.5+	前端核心框架（组合式 API）
构建	vite	6.0+	开发与构建工具
UI 组件	element-plus	2.7+	UI 组件库（按钮、表格、表单等）
状态管理	pinia	3.0+	全局状态管理
路由	vue-router	4.6+	前端路由管理
图表	echarts	5.5+	数据可视化（训练曲线、检测统计）
HTTP	axios	1.7+	HTTP 请求客户端
Markdown	markdown-it	14+	Agent 回复的 Markdown 渲染

二、IDE 安装与配置

2.1 VS Code 安装与配置（推荐）

VS Code 是本项目推荐的 IDE，它轻量、插件丰富，同时对 Python 和 Vue 3 都有 excellent 的支持。

2.1.1 安装 VS Code

- 访问 VS Code 官网：<https://code.visualstudio.com/>
- 下载对应操作系统的安装包（Windows / macOS / Linux）
- 运行安装程序，建议勾选以下选项：

-  Add "Open with Code" action to context menu (右键菜单)
-  Add to PATH (命令行可直接使用 `code` 命令)
-  Register Code as editor for supported file types

2.1.2 必装插件清单

打开 VS Code, 按 `Ctrl+Shift+X` (Windows/Linux) 或 `Cmd+Shift+X` (macOS) 打开插件市场, 搜索安装以下插件:

Python 开发:

插件名称	插件 ID	用途
Python	<code>ms-python.python</code>	Python 语言支持、IntelliSense、调试
Pylance	<code>ms-python.vscode-pylance</code>	Python 语言服务器, 类型检查与补全
Ruff	<code>charliermarsh.ruff</code>	Python 代码格式化与 Lint
autoDocstring	<code>njpwerner.autodocstring</code>	自动生成 Python 文档字符串

Vue / 前端开发:

插件名称	插件 ID	用途
Vue - Official	<code>Vue.volar</code>	Vue 3 语法支持、模板高亮
ESLint	<code>dbaeumer.vscode-eslint</code>	JavaScript 代码检查
Prettier	<code>esbenp.prettier-vscode</code>	前端代码格式化

通用工具:

插件名称	插件 ID	用途
GitLens	<code>eamodio.gitlens</code>	Git 增强, 查看行级 blame 和历史
Git Graph	<code>mhutchie.git-graph</code>	Git 分支可视化
Docker	<code>ms-azuretools.vscode-docker</code>	Dockerfile 支持、容器管理
YAML	<code>redhat.vscode-yaml</code>	YAML 文件语法支持 (可选)
Markdown Preview Enhanced	<code>shd101wyy.markdown-preview-enhanced</code>	Markdown 增强预览 (可选)
Error Lens	<code>usernamehw.errorlens</code>	行内显示错误与警告 (可选)
Path Intellisense	<code>christian-kohler.path-intellisense</code>	文件路径自动补全 (可选)

AI 辅助（可选）：

插件名称	插件 ID	用途
GitHub Copilot	GitHub.copilot	AI 代码补全（学生可免费使用）

一键安装命令（在终端执行）：

```
# Python 相关
code --install-extension ms-python.python
code --install-extension ms-python.vscode-pylance
code --install-extension charliermarsh.ruff
code --install-extension njpwerner.autodocstring

# Vue / 前端相关
code --install-extension Vue.volar
code --install-extension dbaeumer.vscode-eslint
code --install-extension esbenp.prettier-vscode

# 通用工具
code --install-extension eamodio.gitlens
code --install-extension mhutchie.git-graph
code --install-extension ms-azuretools.vscode-docker
code --install-extension redhat.vscode-yaml
code --install-extension shdl01wyy.markdown-preview-enhanced
code --install-extension usernamehw.errorlens
code --install-extension christian-kohler.path-intellisense
```

2.1.3 推荐配置

打开 VS Code 设置（`Ctrl+,` 或 `Cmd+,`），搜索以下配置项，或直接在项目的 `.vscode/settings.json` 中添加：

```
{
  // Python 解释器路径（指向项目虚拟环境）
  "python.defaultInterpreterPath": "${workspaceFolder}/backend/.venv/bin/python",
  "python.analysis.typeCheckingMode": "basic",
  "python.analysis.autoImportCompletions": true,

  // 编辑器设置
  "editor.formatOnSave": true,
  "editor.codeActionsOnSave": {
    "source.organizeImports": "explicit"
  },
  "editor.tabSize": 4,
  "editor.rulers": [120],

  // 不同语言的格式化器
  "[python]": {
    "editor.defaultFormatter": "charliermarsh.ruff",
```

```

    "editor.tabSize": 4
  },
  "[vue]": {
    "editor.defaultFormatter": "esbenp.prettier-vscode",
    "editor.tabSize": 2
  },
  "[javascript]": {
    "editor.defaultFormatter": "esbenp.prettier-vscode",
    "editor.tabSize": 2
  },
},

// 文件排除（不在侧栏显示）
// "files.exclude": {
//   "**/__pycache__": true,
//   "**/.venv": true,
//   "**/node_modules": true,
//   "**/.git": true
// },

// 终端配置
"terminal.integrated.defaultProfile.osx": "zsh",
"terminal.integrated.defaultProfile.linux": "bash",
"terminal.integrated.defaultProfile.windows": "PowerShell"
}

```

2.1.4 配置 Python 虚拟环境

1. 用 VS Code 打开项目文件夹：File → Open Folder → rsod-agent-platform
2. 按 `Ctrl+Shift+P`（或 `Cmd+Shift+P`）打开命令面板
3. 输入 Python: Select Interpreter
4. 选择项目根目录下的 `.venv` 虚拟环境对应的 Python 解释器
5. 打开终端（`Ctrl+``），确认提示符前有 `(.venv)` 标识

2.2 PyCharm 安装与配置（根据个人开发IDE喜好来选择）

PyCharm 是 JetBrains 出品的专业 Python IDE，对 Python 项目有更强大的调试和重构功能。

2.2.1 安装 PyCharm

1. 访问 PyCharm 官网：<https://www.jetbrains.com/pycharm/>
2. 下载版本选择：
 - **Community Edition**（社区版）：免费，满足纯 Python 开发需求
 - **Professional Edition**（专业版）：支持 Web 开发、数据库工具等，学生可免费使用
3. 学生免费申请：访问 <https://www.jetbrains.com/community/education/>，使用学校邮箱注册

2.2.2 配置项目

- 1. 打开 PyCharm，选择 `Open` 打开项目文件夹 `rsod-agent-platform`
- 2. 配置 Python 解释器：
 - `File` → `Settings` → `Project` → `Python Interpreter` (Windows/Linux)
 - `PyCharm` → `Preferences` → `Project` → `Python Interpreter` (macOS)
 - 点击齿轮图标 → `Add...` → 选择 `Virtualenv Environment` → `Existing environment`
 - 指向项目根目录下的 `.venv` 文件夹中的 Python 解释器
- 3. 安装依赖：打开 PyCharm 内置终端（底部 Terminal 标签），执行：

```
pip install -r backend/requirements.txt
```

- 4. 标记源代码目录：右键 `backend` 文件夹 → `Mark Directory as` → `Sources Root`

2.2.3 推荐插件

`Settings` → `Plugins` → `Marketplace` 中搜索安装：

插件名称	用途
Vue.js	Vue 3 语法支持
.env files support	环境变量文件高亮
Rainbow Brackets	彩色括号，提升代码可读性
Key Promoter X	快捷键提示，帮助熟悉操作

三、Docker 与 Docker Compose 部署

本项目使用 Docker Compose 统一管理基础设施服务（PostgreSQL、Redis、MinIO、Ollama），避免在本地逐个安装和配置这些服务。

3.1 Docker Desktop 安装

3.1.1 Windows 安装

前提条件：

- Windows 10（版本 2004 及以上）或 Windows 11
- 需要启用 WSL 2（Windows Subsystem for Linux 2）

安装步骤：

- 1. 启用 WSL 2（如果尚未启用）：
以管理员身份打开 PowerShell，执行：

```
# 启用 WSL
dism.exe /online /enable-feature /featurename:Microsoft-Windows-Subsystem-Linux /all /norestart

# 启用虚拟机平台
dism.exe /online /enable-feature /featurename:VirtualMachinePlatform /all /norestart

# 重启电脑
shutdown /r /t 0
```

重启后，继续执行：

```
# 设置 WSL 2 为默认版本
wsl --set-default-version 2

# 安装默认的 Ubuntu 发行版
wsl --install -d Ubuntu
```

2. 下载并安装 Docker Desktop：

- 访问 <https://www.docker.com/products/docker-desktop/>
- 下载 **Windows - AMD64** 版本
- 运行安装程序，勾选 **Use WSL 2 instead of Hyper-V**
- 安装完成后重启电脑

3. 启动 Docker Desktop：

- 从开始菜单启动 Docker Desktop
- 等待左下角状态变为 **绿色 (Engine running)**
- 首次启动可能需要等待 1-2 分钟

3.1.2 macOS 安装

1. 访问 <https://www.docker.com/products/docker-desktop/>
2. 根据芯片类型下载对应版本：
 - Apple Silicon (M1/M2/M3) → 下载 **Apple Silicon** 版本
 - Intel 芯片 → 下载 **Intel** 版本
3. 打开 `.dmg` 文件，将 Docker 图标拖入 Applications 文件夹
4. 从 Applications 启动 Docker Desktop
5. 等待顶部菜单栏的鲸鱼图标显示 **Docker Desktop is running**

3.1.3 Linux (Ubuntu/Debian) 安装

```
# 卸载旧版本
sudo apt remove docker docker-engine docker.io containerd runc 2>/dev/null
```

```

# 安装依赖
sudo apt update
sudo apt install ca-certificates curl gnupg -y

# 添加 Docker 官方 GPG 密钥
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o
/etc/apt/keyrings/docker.gpg
sudo chmod a+r /etc/apt/keyrings/docker.gpg

# 添加 Docker 仓库
echo \
  "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg]
https://download.docker.com/linux/ubuntu \
  $(. /etc/os-release && echo "$VERSION_CODENAME") stable" | \
  sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

# 安装 Docker Engine + Docker Compose 插件
sudo apt update
sudo apt install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-
compose-plugin -y

# 将当前用户加入 docker 组 (免 sudo 运行 docker)
sudo usermod -aG docker $USER
newgrp docker

```

3.1.4 验证 Docker 安装

```

docker --version
# 期望输出: Docker version 24.x.x 或更高

docker compose version
# 期望输出: Docker Compose version v2.x.x

# 运行 hello-world 测试
docker run hello-world
# 期望输出: Hello from Docker! This message shows that your installation appears to be
working correctly.

```

3.1.5 配置 Docker 镜像加速（国内用户）

Docker Hub 在国内访问可能较慢，配置镜像加速器：

Docker Desktop (Windows / macOS) :

1. 打开 Docker Desktop → Settings（齿轮图标） → Docker Engine
2. 在 JSON 配置中添加 `registry-mirrors`：

```

{
  "builder": {
    "gc": {

```



```

    "defaultKeepStorage": "20GB",
    "enabled": true
  },
  "experimental": false,
  "features": {
    "buildkit": true
  },
  "registry-mirrors": [
    "https://2a6bf1988cb6428c877f723ec7530dbc.mirror.swr.myhuaweicloud.com",
    "https://docker.m.daocloud.io",
    "https://hub-mirror.c.163.com",
    "https://mirror.baidubce.com",
    "https://your_preferred_mirror",
    "https://dockerhub.icu",
    "https://docker.registry.cyou",
    "https://docker-cf.registry.cyou",
    "https://dockercf.jsdelivr.fyi",
    "https://docker.jsdelivr.fyi",
    "https://dockertest.jsdelivr.fyi",
    "https://mirror.aliyuncs.com",
    "https://dockerproxy.com",
    "https://mirror.baidubce.com",
    "https://docker.m.daocloud.io",
    "https://docker.nju.edu.cn",
    "https://docker.mirrors.sjtug.sjtu.edu.cn",
    "https://docker.mirrors.ustc.edu.cn",
    "https://mirror.iscas.ac.cn",
    "https://docker.rainbond.cc"
  ]
}

```

3. 点击 **Apply & Restart**

3.2 Docker Compose 项目服务编排

本项目通过 `docker-compose.yml` 一键启动以下基础设施服务：

服务名	镜像	端口	用途
postgres	pgvector/pgvector:pg15	5432	PostgreSQL 数据库 + Pgvector 向量扩展
redis	redis:7-alpine	6379	缓存 + 对话历史存储
minio	minio/minio:latest	9000 (API) / 9001 (控制台)	对象存储（图片/模型文件）
ollama	ollama/ollama:latest	11434	本地 LLM 服务（可选）

3.2.1 docker-compose.yml 配置

在项目根目录创建 `docker-compose.yml` 文件：

```
version: "3.8"

services:
  # — PostgreSQL + Pgvector —————
  postgres:
    image: pgvector/pgvector:pg15
    container_name: rsod-agent-postgres
    restart: unless-stopped
    environment:
      POSTGRES_DB: rsod_agent
      POSTGRES_USER: rsod_admin
      POSTGRES_PASSWORD: rsod_admin
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U rsod_admin -d rsod_agent"]
      interval: 5s
      timeout: 5s
      retries: 5

  # — Redis —————
  redis:
    image: redis:7-alpine
    container_name: rsod-agent-redis
    restart: unless-stopped
    ports:
      - "6379:6379"
    volumes:
      - redis_data:/data
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 5s
      timeout: 5s
      retries: 5

  # — MinIO 对象存储 —————
  minio:
    image: minio/minio:latest
    container_name: rsod-agent-minio
    restart: unless-stopped
    environment:
      MINIO_ROOT_USER: minioadmin
      MINIO_ROOT_PASSWORD: minioadmin
    ports:
      - "9000:9000" # API 端口
      - "9001:9001" # Console 端口
    volumes:
      - minio_data:/data
```

```

command: server /data --console-address ":9001"
healthcheck:
  test: ["CMD-SHELL", "wget -qO- http://localhost:9000/minio/health/live"]
  interval: 10s
  timeout: 5s
  retries: 3

# — Ollama 本地 LLM (可选) —————
# ollama:
#   image: ollama/ollama:latest
#   container_name: rsod-agent-ollama
#   restart: unless-stopped
#   ports:
#     - "11434:11434"
#   volumes:
#     - ollama_data:/root/.ollama
# GPU 加速 (取消注释以启用 NVIDIA GPU 支持)
# deploy:
#   resources:
#     reservations:
#       devices:
#         - driver: nvidia
#           count: all
#           capabilities: [gpu]

networks:
  default:
    name: rsod-network

volumes:
  postgres_data:
  redis_data:
  minio_data:
  ollama_data:

```

3.2.2 环境变量配置

在后端项目根目录创建 `.env.example` 文件（注意：此文件不包含敏感信息，用于提交到Git，复制此文件重命名为`.env`,用于开发环境，包含敏感信息，`.gitignore` 已配置忽略，将不会提交到 Git）：

```

# =====
# 基于 YOLOv11 的目标检测智能体平台 — 环境变量配置
# =====

# — OpenAI API 配置 (根据项目需求进行修改使用) —————
OPENAI_API_KEY=sk-your-api-key-here
OPENAI_MODEL=gpt-4o-mini
OPENAI_BASE_URL=https://api.openai.com/v1

# — Qwen API 配置 —————
QWEN_API_KEY=sk-your-qwen-api-key

```

```
QWEN_API_URL=https://dashscope.aliyuncs.com/api/v1/services/aigc/text-
generation/generation
# Qwen 兼容模式 API (用于 LangChain OpenAI 客户端)
QWEN_BASE_URL=https://dashscope.aliyuncs.com/compatible-mode/v1

# — 数据库配置 —————
DB_HOST=localhost
DB_PORT=5432
DB_NAME=your-database-name
DB_USER=your-database-username
DB_PASSWORD=your-database-password

# — Redis 配置 —————
REDIS_HOST=localhost
REDIS_PORT=6379

# 开发环境: 本地 Docker
DATABASE_URL=postgresql://your-database-username:your-database-
password@localhost:5432/your-database-name
# 生产环境: 取消下一行的注释并注释掉上面的行
# DATABASE_URL=postgresql://your-database-username:your-secure-password@your-
host:5432/your-database-name

# — 本地 LLM 配置 (可选) —————
USE_LOCAL_LLM=false
OLLAMA_BASE_URL=http://localhost:11434
OLLAMA_MODEL=qwen2.5:7b

# — Redis 配置 —————
# 开发环境: 本地 Docker
REDIS_URL=redis://localhost:6379/0
# 生产环境: 取消下一行的注释并注释掉上面的行
# REDIS_URL=redis://:your-redis-password@your-redis-host:6379/0

# — MinIO 对象存储配置 —————
MINIO_ENDPOINT=localhost:9000
MINIO_ACCESS_KEY=your-minio-access-key
MINIO_SECRET_KEY=your-minio-secret-key
MINIO_BUCKET=rsod-images
MINIO_SECURE=false

# — JWT 认证配置 —————
JWT_SECRET_KEY=your-super-secret-key-change-in-production
JWT_ALGORITHM=HS256
ACCESS_TOKEN_EXPIRE_MINUTES=30

# — 应用配置 —————
APP_NAME=RSOD Agent Platform
APP_VERSION=0.1.0
DEBUG=true
LOG_LEVEL=INFO
```

```
# — CORS 配置 —————  
ALLOWED_ORIGINS=http://localhost:3000,http://localhost:8080
```

⚠️ 重要：确保 `.env` 文件已添加到 `.gitignore`，绝对不要将 API 密钥、密码等敏感信息提交到 Git 仓库。

3.2.3 启动基础设施服务

```
# 确保在项目根目录 (docker-compose.yml 所在目录)  
cd rsod-agent-platform  
  
# 拉取镜像 (首次运行需要, 可能耗时较长)  
docker compose pull  
  
# 启动所有服务 (后台运行)  
docker compose up -d  
  
# 查看服务状态  
docker compose ps
```

正常输出应类似：

NAME	IMAGE	STATUS
rsod-postgres	pgvector/pgvector:pg15	Up (healthy)
rsod-redis	redis:7-alpine	Up (healthy)
rsod-minio	minio/minio:latest	Up (healthy)
# rsod-ollama	ollama/ollama:latest	Up

3.2.4 验证各服务是否正常

```
# 1. 验证 PostgreSQL  
docker compose exec postgres psql -U rsod_user -d rsod_agent -c "SELECT version();"   
  
# 2. 验证 Redis  
docker compose exec redis redis-cli ping  
# 期望输出: PONG  
  
# 3. 验证 MinIO — 浏览器访问  
# 打开 http://localhost:9001  
# 用户名: minioadmin 密码: minioadmin  
  
# 4. 验证 Ollama (如果启动了)  
curl http://localhost:11434/api/tags
```

3.3 Docker Compose 常用命令

```
# 启动所有服务 (后台运行)
```

```
docker compose up -d

# 启动指定服务
docker compose up -d postgres redis minio

# 查看服务状态
docker compose ps

# 查看服务日志
docker compose logs -f                # 所有服务
docker compose logs -f postgres      # 指定服务

# 停止所有服务（保留数据）
docker compose down

# 停止并删除数据卷（⚠️ 会清除所有数据！慎用！）
docker compose down -v

# 重启某个服务
docker compose restart redis

# 进入容器内部
docker compose exec postgres psql -U rsod_user -d rsod_agent
docker compose exec redis redis-cli
docker compose exec minio mc alias set local http://localhost:9000 minioadmin minioadmin

# 查看资源占用
docker stats
```

四、项目初始化与运行

当基础环境全部安装完成后，按以下步骤启动整个项目：

4.1 创建main.py

```
from fastapi import FastAPI

app = FastAPI(
    title="RSOD Agent Platform",
    version="0.1.0",
    description="基于 YOLOv11 的目标检测智能体平台 API",
    docs_url="/docs",
    redoc_url="/redoc"
)

@app.get("/")
def root():
    return {
        "message": "欢迎使用 RSOD Agent Platform",
        "version": "0.1.0",
    }
```

```

        "docs": "/docs",
        "redoc": "/redoc"
    }

@app.get("/api/health")
def health_check():
    return {
        "status": "healthy",
        "app_name": "RSOD Agent Platform",
        "version": "0.1.0"
    }

@app.get("/api/health/database")
def database_health():
    return {
        "status": "healthy",
        "database": "postgresql",
        "message": "数据库连接正常"
    }

@app.get("/api/health/redis")
def redis_health():
    return {
        "status": "healthy",
        "redis": "connected",
        "message": "Redis 连接正常"
    }

@app.get("/api/health/minio")
def minio_health():
    return {
        "status": "healthy",
        "minio": "connected",
        "message": "MinIO 连接正常"
    }

if __name__ == "__main__":
    import uvicorn
    uvicorn.run("main:app", host="0.0.0.0", port=8000, reload=True)

```

4.2 启动基础设施服务

```

# 进入项目根目录
cd rsod-agent-platform
# 启动 PostgreSQL、Redis、MinIO 等
docker compose up -d

```

4.3 启动后端服务

```
# 创建并激活虚拟环境
python -m venv .venv
source .venv/bin/activate      # macOS / Linux
.venv\Scripts\activate        # Windows

# 安装依赖（未安装的情况下）
pip install -r requirements.txt

# 配置环境变量
cp .env.example .env          # 如果有 .env.example

# 启动 FastAPI 后端服务
uvicorn backend.main:app --reload --host 0.0.0.0 --port 8000
或
python main.py
```

启动成功后访问：

- **API 文档：** <http://localhost:8000/docs>（Swagger UI 交互式文档）
- **备用文档：** <http://localhost:8000/redoc>（ReDoc 文档）

4.4 前端环境配置（Vue3 + Vite 项目初始化）

```
# 新开一个终端,进入项目根目录, 创建Vite+Vue项目(默认创建完会自动启动 Vite 开发服务器)
npm create vite@latest frontend -- --template vue
# 进入前端目录
cd frontend

# 安装依赖
npm install

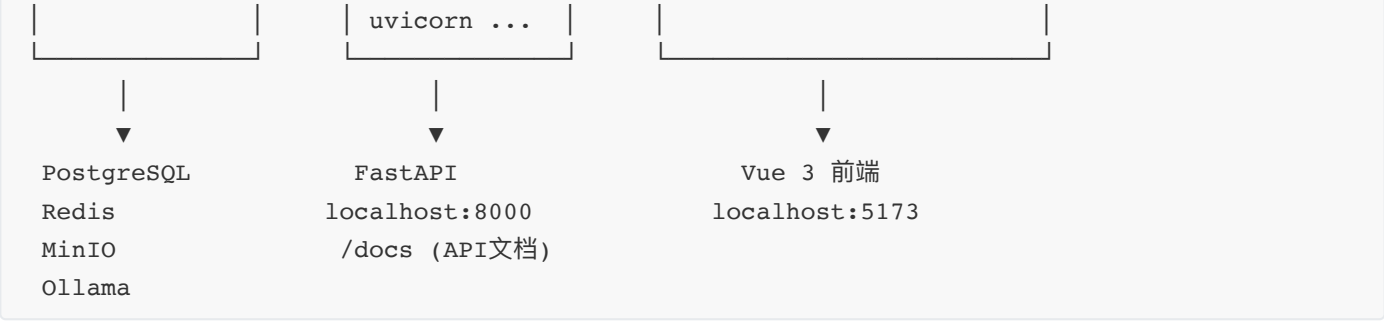
# 启动 Vite 开发服务器
npm run dev
```

启动成功后访问：

- **前端页面：** <http://localhost:5173>

4.5 完整启动流程总结

终端 1	终端 2	终端 3
docker compose up -d	cd backend source .venv/bin/ activate	cd frontend npm install npm run dev



五、环境验证清单

完成所有安装后，逐项执行以下命令验证环境：

```
# =====
# 基础工具版本检查
# =====

git --version
# ✅ git version 2.x.x

python --version          # 或 python3 --version
# ✅ Python 3.10.x 或 3.11.x

node --version
# ✅ v24.14.0

npm --version
# ✅ 11.9.0 或更高

docker --version
# ✅ Docker version 28.1.1 或更高

docker compose version
# ✅ Docker Compose version v2.x.x

# =====
# SSH 连接检查
# =====

ssh -T git@github.com
# ✅ Hi xxx! You've successfully authenticated...

# =====
# Python 虚拟环境检查
# =====

# 确保终端提示符前有 (.venv) 标识
python -c "import fastapi; print('✓ FastAPI')"
python -c "import uvicorn; print('✓ Uvicorn')"
python -c "import langchain; print('✓ LangChain')"
```

```
python -c "import langgraph; print('✓ LangGraph')"
python -c "import ultralytics; print('✓ Ultralytics (YOLOv11)')"
python -c "import cv2; print('✓ OpenCV')"

# =====
# Docker 服务检查
# =====

docker compose ps
# ✓ 所有服务状态为 Up / healthy

docker compose exec redis redis-cli ping
# ✓ PONG

# 浏览器可访问:
# ✓ http://localhost:9001 - MinIO 控制台
# ✓ http://localhost:8000/docs - FastAPI API 文档 (后端启动后)
# ✓ http://localhost:5173 - Vue 前端页面 (前端启动后)
```

全部通过后，恭喜！开发环境搭建完成。 🎉

六、常见问题排查

Q1: Windows 下 `python` 命令提示 "不是内部或外部命令"

原因：安装 Python 时未勾选 "Add Python to PATH"。

解决：

1. 打开 Windows 设置 → 搜索"环境变量"
2. 点击"编辑系统环境变量" → "环境变量"
3. 在"Path"中添加 Python 安装路径，例如：

```
C:\Users\你的用户名\AppData\Local\Programs\Python\Python310\
C:\Users\你的用户名\AppData\Local\Programs\Python\Python310\Scripts\
```

4. 重启终端

或者直接重新运行 Python 安装程序，勾选 "Add Python to PATH"。

Q2: Windows PowerShell 激活虚拟环境时报错 "无法加载文件...因为在此系统上禁止运行脚本"

解决：以管理员身份打开 PowerShell，执行：

```
Set-ExecutionPolicy RemoteSigned -Scope CurrentUser
```

输入 `y` 确认，然后重新激活虚拟环境。

Q3: pip 安装包速度很慢

解决：配置国内镜像源（参见 [1.2.3 节](#)）。

Q4: Docker Desktop 启动失败（Windows）

常见原因：WSL 2 未正确安装或未更新。

解决：

```
# 以管理员身份运行 PowerShell
wsl --update
wsl --set-default-version 2

# 如果仍然失败，尝试重启电脑后再启动 Docker Desktop
```

Q5: Docker Compose 启动时端口被占用

例如 `Bind for 0.0.0.0:5432 failed: port is already allocated`。

解决：

```
# 查看哪个进程占用了端口
# Windows:
netstat -ano | findstr :5432

# macOS / Linux:
lsof -i :5432

# 方案 1: 停止占用端口的程序
# 方案 2: 修改 docker-compose.yml 中的端口映射，如 5432:5432 改为 15432:5432
```

Q6: npm install 报错 "EACCES permission denied"（macOS / Linux）

解决：不要使用 `sudo npm install`，而是修复 npm 目录权限：

```
mkdir -p ~/.npm-global
npm config set prefix '~/.npm-global'
echo 'export PATH=~/.npm-global/bin:$PATH' >> ~/.zshrc # macOS
echo 'export PATH=~/.npm-global/bin:$PATH' >> ~/.bashrc # Linux
source ~/.zshrc # 或 source ~/.bashrc
```

Q7: ultralytics 安装失败或 import 报错

常见原因：系统缺少编译工具或依赖库。

解决：

```
# Windows: 安装 Visual C++ Build Tools
# 下载: https://visualstudio.microsoft.com/visual-cpp-build-tools/
# 安装时勾选 "C++ build tools"

# macOS: 安装 Xcode Command Line Tools
xcode-select --install

# Linux: 安装编译工具
sudo apt install build-essential python3-dev -y
```

Q8: Ollama 模型下载很慢或失败（备选，可不下载）

Ollama 需要下载 LLM 模型文件（通常几个 GB），国内网络可能较慢。

解决方案：

1. 暂时不启动 Ollama 服务，使用 OpenAI API 作为替代
2. 在 `.env` 中设置 `USE_LOCAL_LLM=false`
3. 在 `docker-compose.yml` 中注释掉 `ollama` 服务

```
# ollama:
#   image: ollama/ollama:latest
#   ...
```

Q9: MinIO 控制台无法访问

排查步骤：

```
# 1. 确认 MinIO 容器正在运行
docker compose ps minio

# 2. 查看 MinIO 日志
docker compose logs minio

# 3. 确认访问的是控制台端口 9001（不是 API 端口 9000）
# 浏览器访问: http://localhost:9001
```

附录：环境版本汇总表

工具 / 技术	最低版本	推荐版本	验证命令
Git	2.30+	2.43+	<code>git --version</code>
Python	3.10+	3.10.11 / 3.11.9	<code>python --version</code>
Node.js	18+	18.19+ / 20.11+	<code>node --version</code>
npm	9+	10.2+	<code>npm --version</code>
Docker	20.10+	24.0+	<code>docker --version</code>
Docker Compose	2.0+	2.23+	<code>docker compose version</code>
PostgreSQL	15+	15.x (pgvector 镜像)	Docker 容器内运行
Redis	7+	7.x Alpine	Docker 容器内运行

文档版本：v1.0
适用项目：基于 YOLOv11 的目标检测智能体平台
最后更新：2026-06-27